CS2109S: Introduction to AI and Machine Learning

Lecture 6:
Logistic Regression

17 February 2026

Announcements

Midterm – Reminder

- Date & Time:
  - **Monday, 2nd March 2026**, from **18:30** to 20:30 (18:15 entry)
  - No additional time will be provided for latecomers.
- Venue:
  - **MPSH 1A & 1B**
- Format:
  - Digital Assessment (**Examplify**)
- Materials:
  - All topics covered **until and including Lecture 6**
- Cheatsheet:
  - **1 x A4 paper, both sides**
- Calculators:
  - Standard and scientific calculators are allowed.
  - Graphing or programmable calculators are allowed, but please make sure they do not contain any stored functions or programs.

More details will be announced later.

Midterm – Coverage

- **Intelligent agents**
- **Uninformed search**: problem formulation, BFS, DFS, UCS, DLS, IDS, visited memory.
- **Informed search**: A* search, heuristics, admissibility, consistency, dominance.
- **Local search**: problem formulation, hill-climbing.
- **Adversarial search**: game tree, minimax, alpha-beta pruning, cutoff.
- **ML, Supervised learning**: data, model, performance measure and loss, learning algorithms
- **Decision trees**: model, decision tree learning, pruning.
- **Linear regression**: linear model, loss, gradient descent, normal equation, feature transformations.
- **Logistic regression**: logistic model, loss, multi-class and label classification, generalization.

Basically, materials covered up to week 6

Schedule: Recess Week & Midterm

- This week: CNY
  - Lecture 6: Online lecture recording.
  - Tutorials recorded/online/offline.
- Next week: Recess Week, no tutorial
- Week 7: has Midterm, no tutorial
- Week 8: Lecture 7, has tutorial

Lecture

Outline

- **Logistic Regression**
  - Data
  - Model
  - Loss
- Learning via Gradient Descent
- Feature Transformation
- Multi-Class Classification
- Multi-Label Classification
- Advanced Topics in Supervised Learning
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Example: Engine Failure Prediction

Given an engine temperature, will the engine fail?

Temperature	Failure?
20	False
50	False
95	False
120	True
190	True
…	…

Data

Suppose:

- We are given n data points.
- Each data point consists of features and a target variable.
- Each data point has d features and they are described by real numbers.
- The target is {0,1}, where 0 is “negative” class and 1 is “positive” class.

Data – Math

Suppose:

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\},$$

where for all $i \in \{1, \dots, N\}$

Features: $x^{(i)} \in \mathbb{R}^d$

Targets: $y^{(i)} \in \{0,1\}$

Task

Suppose we are given another data point $x \in \mathbb{R}^d$ and no target. Based on the dataset, find a function that predicts the target $y \in \{0,1\}$ for that x.

This task is called classification.

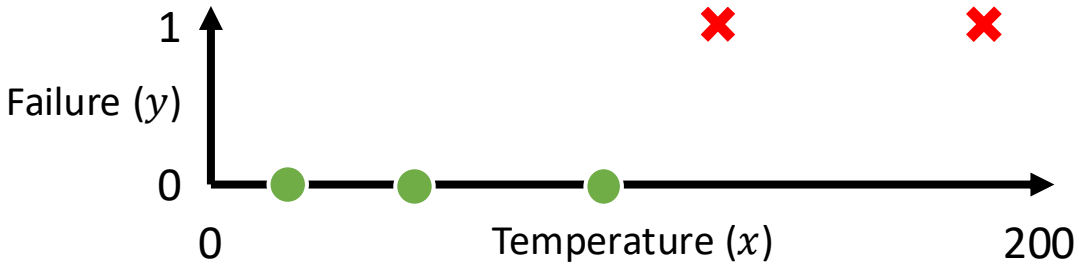
Since the target has only two classes, this is called **binary classification**.

Example: Engine Failure Prediction

Given an engine temperature, will the engine fail?

i	$x^{(i)}$	$y^{(i)}$
1	20	0
2	50	0
3	95	0
4	120	1
5	190	1
…	…	…

Dataset D



Example:

x = 150°C

Will engine fail at x?

What kind of functions should we use?

"squashed" linear functions: $\mathbb{R}^d \rightarrow [0,1]$

Background: Sigmoid Function

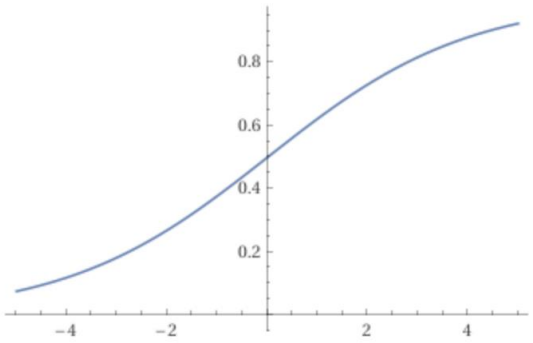
The sigmoid function is a mathematical function that maps a real number to a value between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

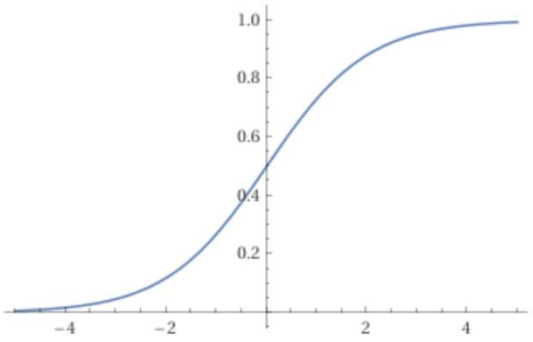
This function is differentiable, with derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

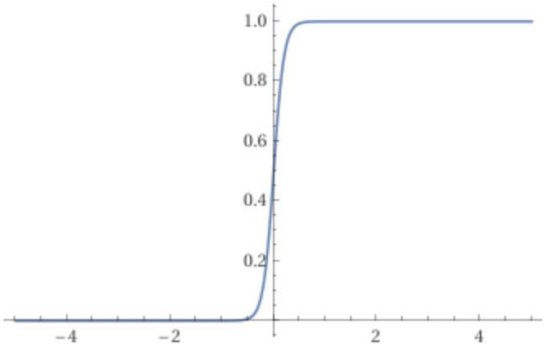
Also known as the **logistic function**.



$\sigma(0.5\,x)$



$\sigma(x)$



$\sigma(10\,x)$

Logistic Model

Given an input vector x of dimension d, a logistic model is a function with the following form:

$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d)$$

where $w_0, \ldots, w_d$ are **parameters/weights** and $x_0 = 1$ is a dummy variable.

We shorthand this function by using the dot product:

$$h_w(x) = \sigma(w^T x) = \sigma\left(\sum_{j=0}^{d} w_j x_j\right)$$

Background: Probability

- **Probability** $P(X)$: A measure of the likelihood that an event X will occur. It ranges from 0 (impossible) to 1 (certain). Example:
  - $P(\text{Failure} = true) = 0.01$
    The probability of engine failure (i.e., that engine failure is true) is 1%.

- **Conditional probability** $P(Y|X)$ is the probability of an event Y occurring given that an event X has already occurred. Example:
  - $P(\text{Failure} = true|\text{Temperature} = 90) = 0.8$
    The probability of engine failure given that the temperature is 90° C is 80%.

Classification with Logistic Model

Logistic model outputs a real number in range [0,1]. The model's output can be treated as the probability of an input to be of class 1.

- This gives the confidence score of the model
- Example:
  - Model's output: $h_w(x) = P(\text{Failure} = true|x) = 0.8$
  - Explanation: The model predicts that the probability of engine failure is 80%

To decide whether an input belongs to a certain class, compare the probability output to a decision threshold τ, i.e., class 1 if $P(\text{y}|x) \geq \tau$ else class 0.

- Example: suppose $\tau = 0.5$, $P(\text{Failure} = true|x) = 0.8$, then it is class 1 (fail)

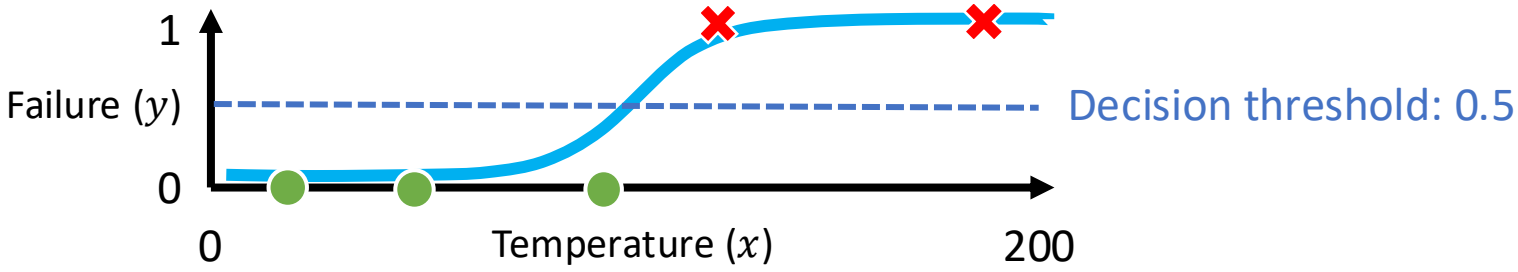
Example: Engine Failure Prediction

$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1)$$

$x_0 = 1$ (dummy variable)

Dataset D

i	$x^{(i)}$	$y^{(i)}$
1	20	0
2	50	0
3	95	0
4	120	1
5	190	1
…	…	…



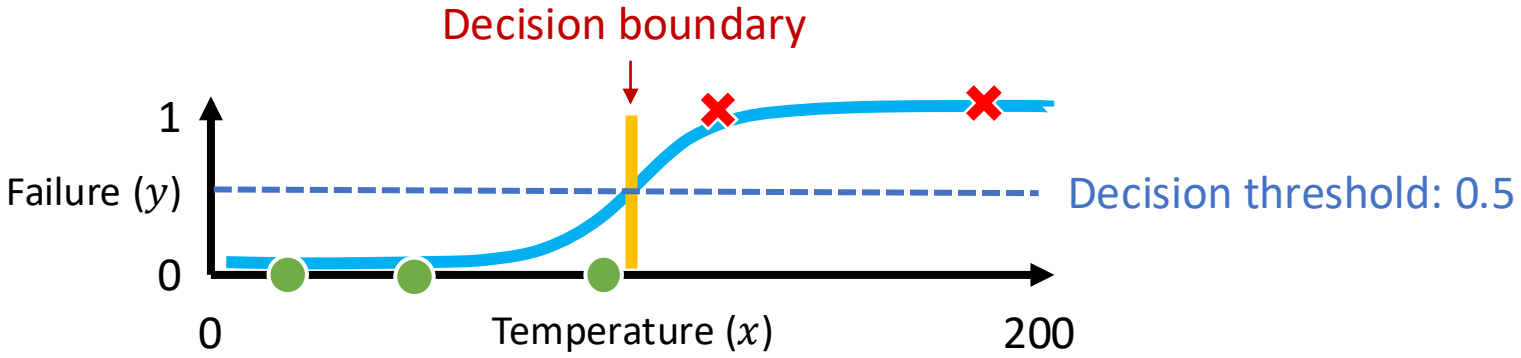
$$h_w(x) = \sigma(-100 + 1x_1)$$

We can set the threshold to 0.5:

- When $h_w(x) \geq 0.5$, classification = 1 (fail)
- When $h_w < 0.5$, classification = 0 (not fail)

Decision Boundary

- A **decision boundary** is a surface (or line in two dimensions, or hyperplane in >2 dimensions) that separates the different classes in the feature space.
- It is an intersection between the **model's function** and the **decision threshold**.
- It defines the points where the classification model changes its predicted class from one to another.
  - For example, in a binary classification task, the decision boundary divides the feature space into two regions, each corresponding to one of the two classes.



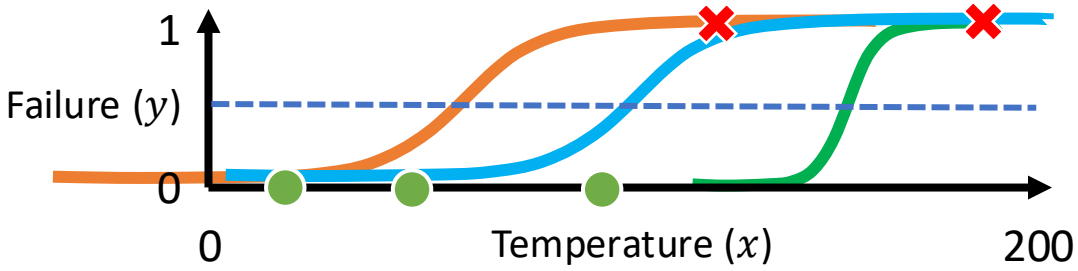
The Best Logistic Model

Déjà vu: There are **infinitely many** logistic models that could represent the relationship between input x and output.

We want a model with the lowest **loss** on the data.

It is common to use **mean squared error** (MSE) loss for linear regression.

Can we use MSE for logistic regression?



$$h_w(x) = \sigma(-10 + 0.1x_1)$$

$$h_w(x) = \sigma(-5 + 0.1x_1)$$

$$h_w(x) = \sigma(-75 + 0.5x_1)$$

Recall: Background: Convexity

Definition: Consider a real-valued one-dimensional function and any line segment connecting any **two distinct points** on the function's graph.

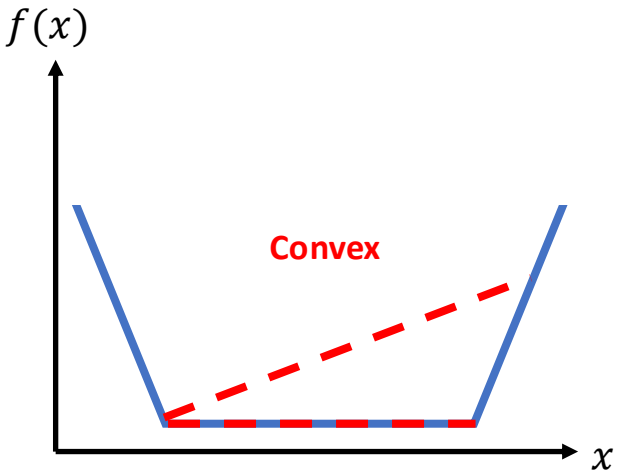
The function is called:

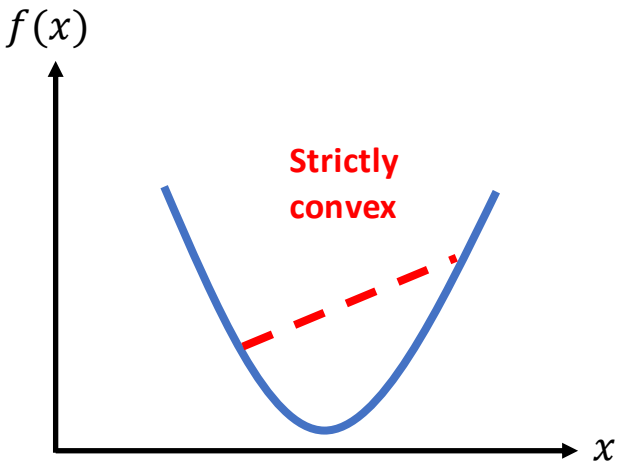
- **Convex** if the line lies above or on the graph between the two points.
- **Strictly convex** if the line always lies above the graph between the two points.

For a two-dimensional function, imagine a bowl-shaped landscape.

Theorem: For a convex function, any local minimum is also a global minimum.

Theorem: A strictly convex function has at most one unique global minimum.





Logistic Regression with MSE Loss – Analysis

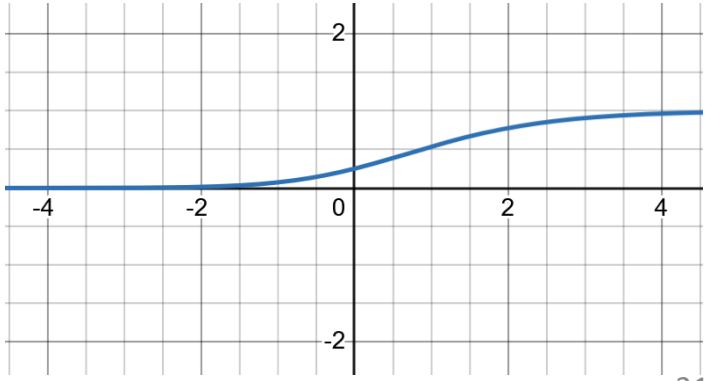
Recall (MSE): Given a dataset $\mathrm{D} = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \mathbb{R}$,

$$J_{MSE}(w) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_w(x^{(i)}) - y^{(i)}\right)^2$$

Theorem: MSE loss function is non-convex for logistic regression.

Proof Idea:

- Logistic function $\sigma(x) = \frac{1}{1+e^{-x}}$ is non-convex.
- MSE squares the function $\sigma(x)$, but it doesn't make it convex –it retains the "S" shape.



$(\sigma(x))^2$

Background: Cross-Entropy

Cross-entropy measures the **difference between two probability distributions** over the same set of outcomes.

Suppose that you have:

- The "true" probability distribution P that represents the **actual** probability of the outcomes
- A probability distribution Q, e.g., **prediction** of a model regarding the probability of the outcomes.

The cross-entropy formula is

$$H(P,Q) = -\sum_{x} P(x)\log(Q(x))$$

Background: Binary Cross-Entropy

Binary Cross-Entropy is a special case of the cross-entropy formula, applied to a situation where there are only **two possible outcomes:** class 1 (positive) and class 0 (negative).

$$H(P,Q) = -\sum_{x} P(x)\log\big(Q(x)\big)$$

$$= -P(1)\log\big(Q(1)\big) - P(0)\log\big(Q(0)\big)$$

The "true" distribution P can be described by a single value y, the true label (1 or 0).

The predicted distribution Q can be described by a single value p, which is the model's predicted probability that the outcome is **class 1**. The probability for class 0 is $1-p$.

$$BCE(y,p) = -y\log(p) - (1-y)\log(1-p)$$

Hint: $\log(1) = 0, \lim_{x\to 0}\log(x) = -\infty$

Binary Cross-Entropy (BCE) Loss

Given a dataset $\mathrm{D} = \left\{\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(n)}, y^{(n)}\right)\right\}$ where $y^{(i)} \in \{0,1\}$,

$$J_{BCE}(w) = \frac{1}{n}\sum_{i=1}^{n} BCE\left(y^{(i)}, h_w\left(x^{(i)}\right)\right)$$

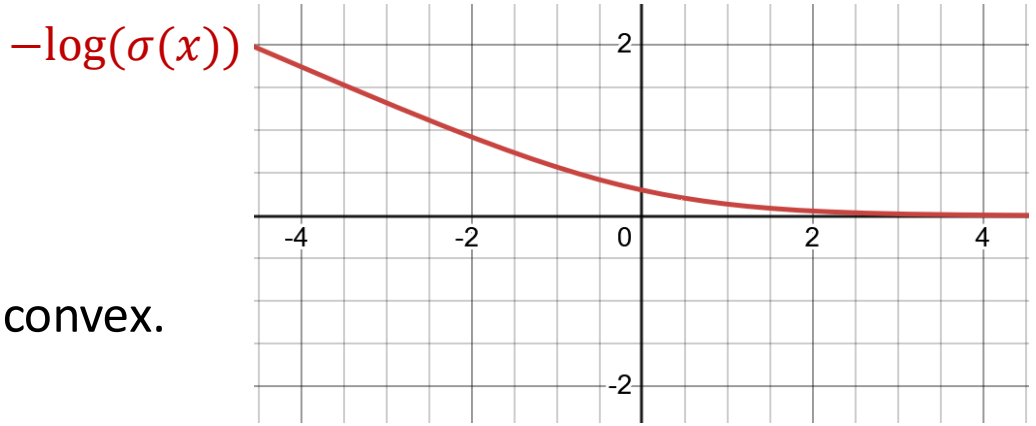
- Logistic regression = logistic model + BCE loss.
- The loss is also known as logistic loss (log loss).
- Notice that J_{BCE} is a function of w; we want to find w that minimizes the loss!

Logistic Regression with BCE Loss

Theorem: BCE loss function for a logistic model is a convex function with respect to the model's parameters.

Proof Idea:

- Logistic function $\sigma(x) = \frac{1}{1+e^{-x}}$ is non-convex.
- However, $-\log$ (i.e., main component of BCE) of $\sigma(x)$ is convex.



Impossibility of Learning via Normal Equation

Model:

$$h_w(x) = \sigma(w_0 + w_1 x_1 + w_2 x_2) = \frac{1}{1 + e^{-w^T x}}$$

Loss Function:

$$J_{BCE}(w) = \frac{1}{n}\sum_{i=1}^{n} BCE(y^{(i)}, h_w(x^{(i)}))$$

Partial Derivative:

$$\frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{\partial}{\partial w_j}\frac{1}{n}\sum_{i=1}^{n} BCE(y^{(i)}, h_w(x^{(i)}))$$

$$= \frac{1}{n}\sum_{i=1}^{n}(h_w(x^{(i)}) - y^{(i)})\, x_j^{(i)}$$

← **Same as Linear Regression**
Derivation will be discussed in the tutorial.

$$0 = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{1}{1 + e^{-w^T x^{(i)}}} - y^{(i)}\right) x_j^{(i)}$$

← **Transcendental equation**
It does not have a general closed-form solution.
Normal equation is not possible.

Outline

- Logistic Regression
  - Data
  - Model
  - Loss
- **Learning via Gradient Descent**
- Feature Transformation
- Multi-Class Classification
- Multi-Label Classification
- Advanced Topics in Supervised Learning
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Recall: Gradient Descent

- Start at some w (e.g., randomly initialized).
- Update w with a step in the opposite direction of the gradient (i.e., towards lower loss)

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1, \dots)}{\partial w_j}.$$

Learning Rate

- Learning rate $\gamma > 0$ is a hyperparameter that determines the step size.
- Repeat until termination criterion is satisfied.
  - E.g., change between steps is small, maximum number of steps is reached, etc.

Logistic Regression with Gradient Descent

Model:

$$h_w(x) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$$

Loss Function:

$$J_{BCE}(w) = \frac{1}{n}\sum_{i=1}^{n} BCE(y^{(i)}, h_w(x^{(i)}))$$

Partial Derivative:

$$\frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{\partial}{\partial w_j}\frac{1}{n}\sum_{i=1}^{n} BCE(y^{(i)}, h_w(x^{(i)}))$$

$$= \frac{1}{n}\sum_{i=1}^{n}(h_w(x^{(i)}) - y^{(i)})\, x_j^{(i)}$$

Weight Update:

$$w_j \leftarrow w_j - \gamma \frac{\partial J_{BCE}(w_0, w_1, \ldots)}{\partial w_j}$$

LR with Gradient Descent – Analysis

Theorem:

For a **logistic model** with a **Binary Cross-Entropy (BCE)** loss function, the **Gradient Descent** algorithm is guaranteed to converge to a **global minimum**, provided the learning rate is chosen appropriately.

Similar to linear regression.

Poll Everywhere

Select the correct statements:

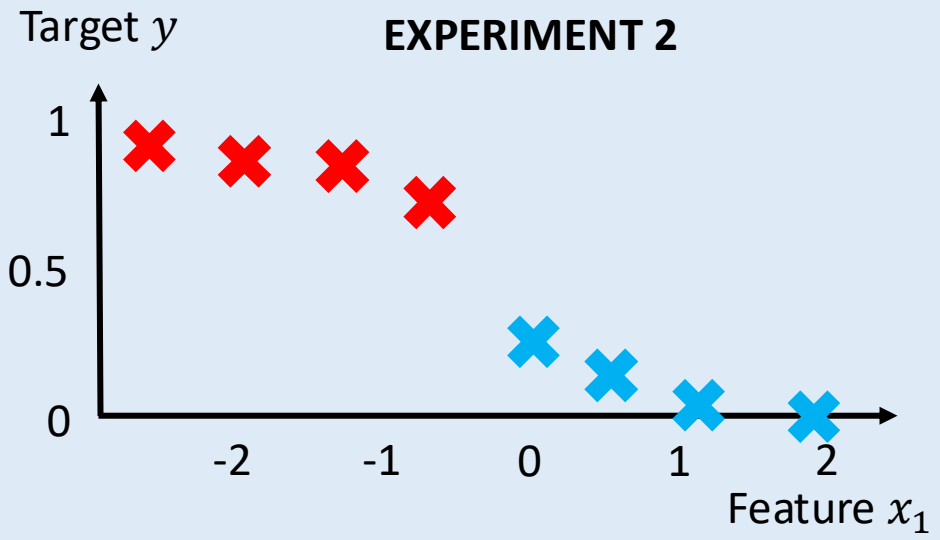
A. Logistic regression is mostly used when the **features** are **categorical** variables.

B. In logistic regression, MSE minimization will find a global minimum solution.

C. For EXPERIMENT 2, the targets $y \in [0,1]$ prevent us from using logistic regression.

D. For EXPERIMENT 2, the best logistic model will have $\boldsymbol{w_1 > 0}$.

E. None of the above.



Poll Everywhere

Select the correct statements:

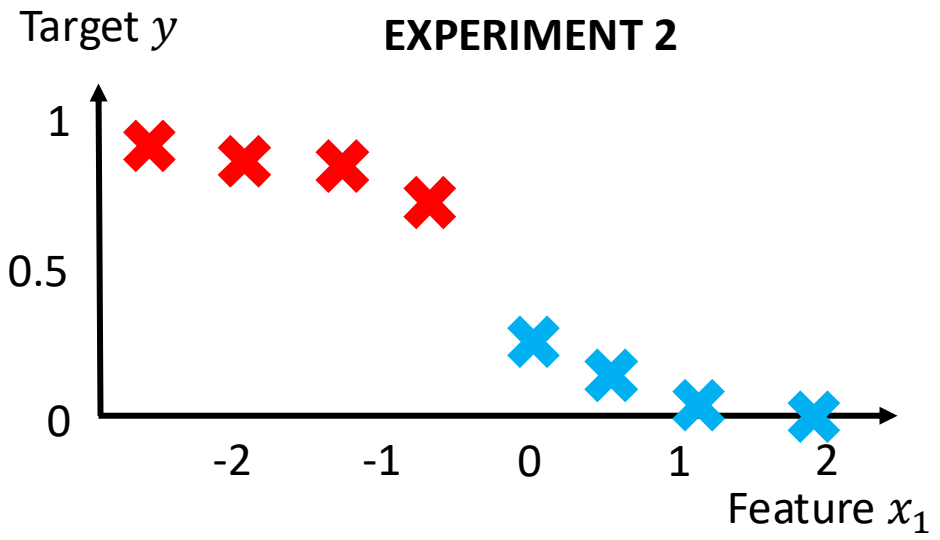
A. Logistic regression is mostly used when the **features** are ~~**categorical**~~ real-valued variables.

B. In logistic regression, ~~MSE~~ BCE minimization will find a global minimum solution.

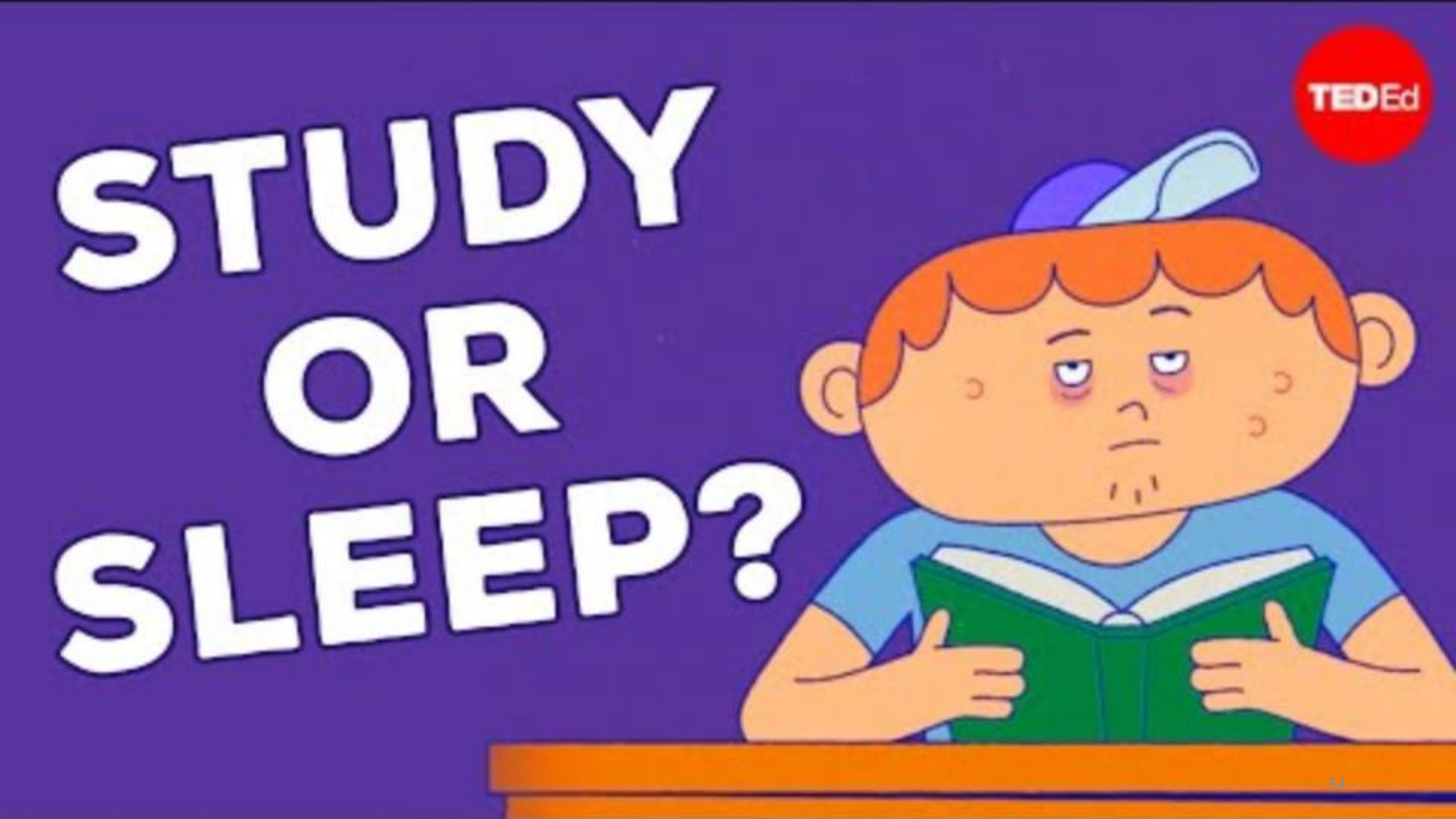
C. For EXPERIMENT 2, the targets $y \in [0,1]$ prevent us from using logistic regression. No issue, BCE computes distance between two probability distributions.

D. For EXPERIMENT 2, the best logistic model will have ~~$\boldsymbol{w_1 > 0}$~~ $\boldsymbol{w_1 < 0}$. Think of $\sigma(-x)$.

E. None of the above.



Break


STUDY
OR
SLEEP?
TEDEd

Outline

- Logistic Regression
  - Data
  - Model
  - Loss
- Learning via Gradient Descent
- **Feature Transformation**
- Multi-Class Classification
- Multi-Label Classification
- Advanced Topics in Supervised Learning
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Example: Engine Failure Prediction

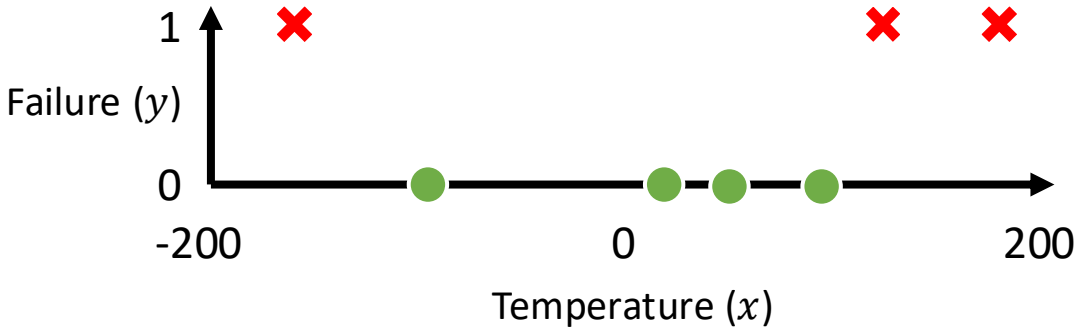
with low-temperature failures

$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1)$$

$x_0 = 1$ (dummy variable)

Dataset D

i	$x^{(i)}$	$y^{(i)}$
1	20	0
2	50	0
3	95	0
4	120	1
5	190	1
6	-100	0
7	-170	1
…	…	…



Data that are Not Linearly Separable

- Data is said to be **linearly separable** when a single linear decision boundary (like a straight line in 2D or a hyperplane in higher dimensions) separates the different classes.

- When this is not the case, the decision boundary required to separate the classes needs to be non-linear or more complex.
  - Use non-linear (more complex) models.
  - Use **non-linear feature transformations.** Example: polynomial, log, exp.

Recall: Feature Transformation

We can take a d-dimensional feature vector and transform it into a M-dimensional feature vector. Usually $M \geq d$.

$$x \in \mathbb{R}^d \quad \rightarrow \quad \phi(x) \in \mathbb{R}^M$$

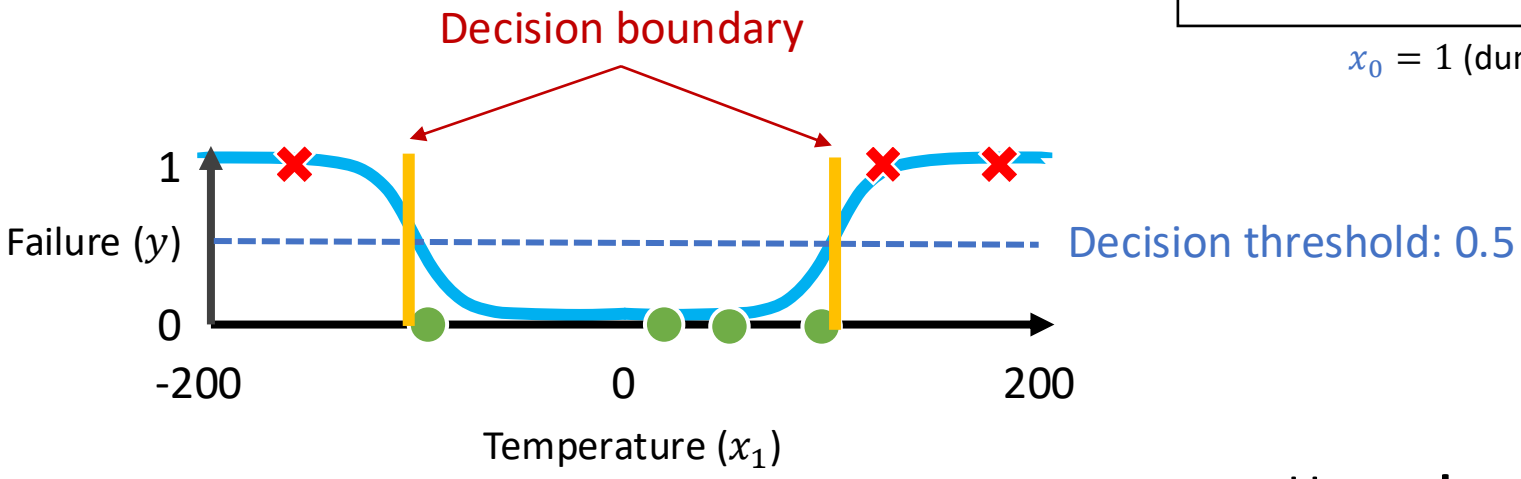
Here,

- The function ϕ is called a feature transformation or feature map,
- The vector $\phi(x)$ is called transformed features.

Logistic Model with Feature Transformation

Dataset D

i	$x_1^{(i)}$	$y^{(i)}$
1	20	0
2	50	0
3	95	0
4	120	1
5	190	1
6	-100	0
7	-170	1
…	…	…



x_1	-200	-100	0	100	200
x_2	40,000	10,000	0	10,000	40,000

$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2)$$

$x_0 = 1$ (dummy variable)

Use **polynomial feature** transformation degree 2:

$$\phi_{p2}([x_1]^T) \rightarrow [x_1 \; x_1^2]^T$$

Logistic model:

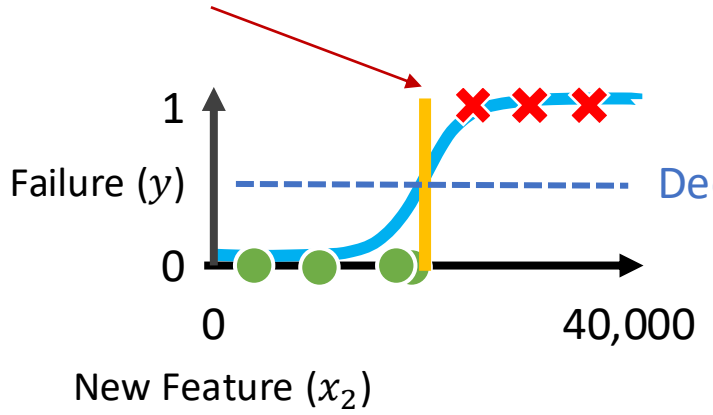
$$h_w(\phi_{p2}(x)) = \sigma\left(-5 + \frac{20}{40{,}000} x_2\right)$$

Logistic Model with Feature Transformation

Dataset D

i	$x_2^{(i)}$	$y^{(i)}$
1	400	0
2	2500	0
3	9025	0
4	14400	1
5	36000	1
6	10000	0
7	28900	1
…	…	…

There is only one Decision boundary



$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2)$$

$x_0 = 1$ (dummy variable)

x_1	-200	-100	0	100	200
x_2	40,000	10,000	0	10,000	40,000

Use **polynomial feature** transformation degree 2:

$$\phi_{p2}([x_1]^T) \rightarrow [x_1 \; x_1^2]^T$$

Logistic model:

$$h_w(\phi_{p2}(x)) = \sigma\left(-5 + \frac{20}{40{,}000} x_2\right)$$

LM with Feature Transformation – Analysis

Theorem: The convexity of logistic model with BCE is preserved **with feature transformation**.

Similar to linear model.

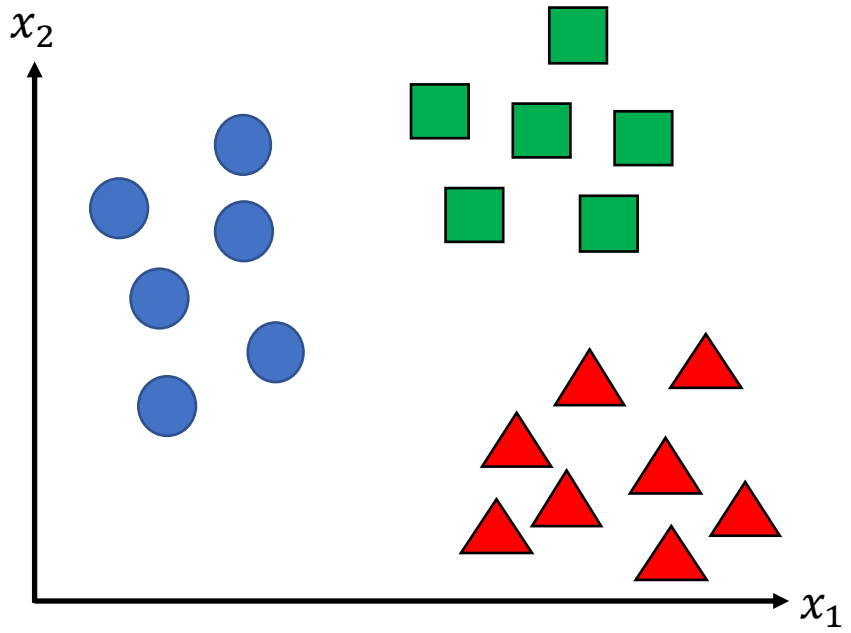
Outline

- Logistic Regression
  - Data
  - Model
  - Loss
- Learning via Gradient Descent
- Feature Transformation
- **Multi-Class Classification**
- Multi-Label Classification
- Advanced Topics in Supervised Learning
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Example: Animal Prediction

Given a set of features describing an animal (e.g., x_1 weight, x_2 height),

Predict whether it is a cat, dog, or rabbit.



Multi-class Classification

Suppose:

- We are given n data points.
- Each data point consists of features and a target variable.
- Each data point has d features and they are described by real numbers.
- The target is {1, 2, …, K} where K is the number of classes.

Suppose we are given another data point $x \in \mathbb{R}^d$ and no target. Based on the dataset, find a model that predicts the target $y \in \{1,2, \dots, K\}$ for that x.

How to do this if we only have binary classifiers?

Binary to Multi-class Classification

Converting binary classification into **multi-class classification** involves breaking down multi-class classification problem into a set of binary classification problems.

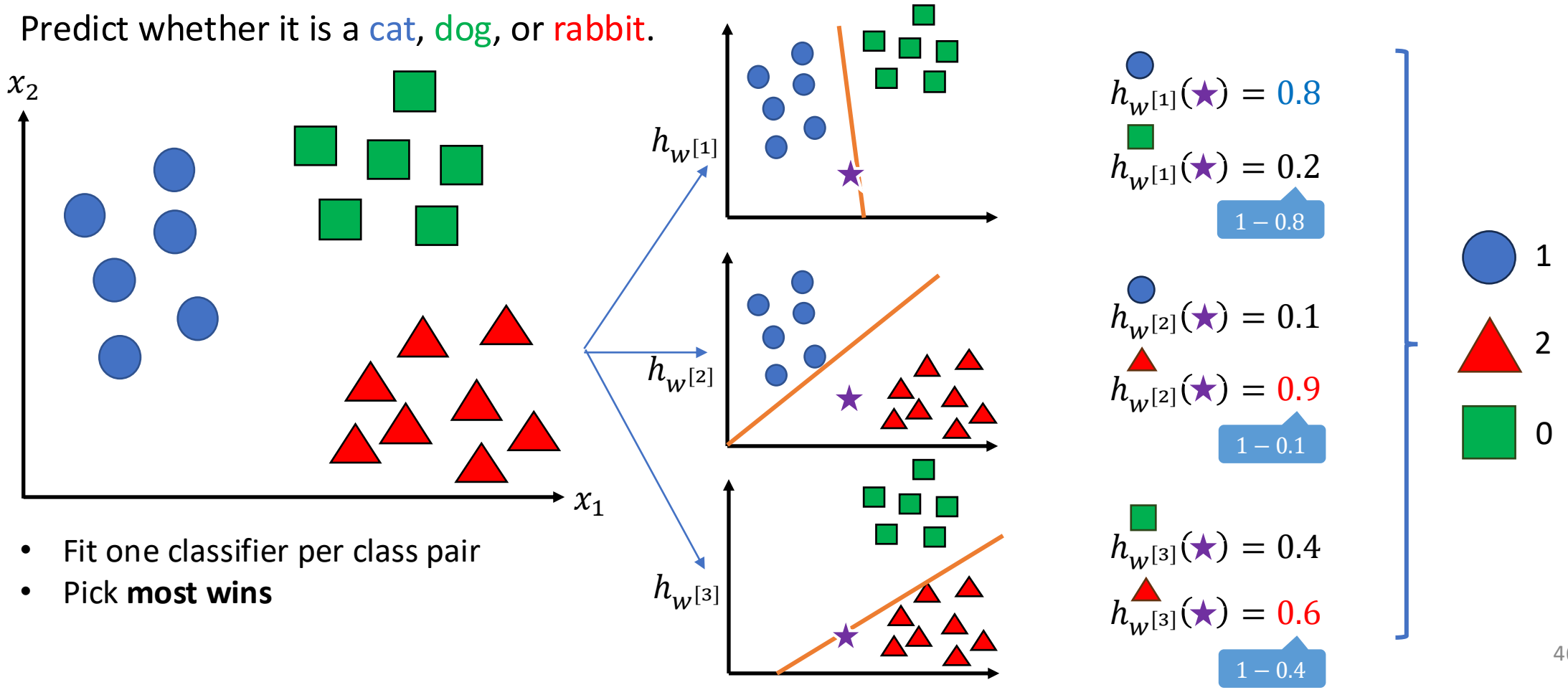
	One-vs-One	One-vs-Rest
Binary Classifier	For **every pair of classes**.	For each class, **treating all other classes as a single combined class**
Number of binary classifiers	$C(K,2) = \frac{K(K-1)}{2}$	K
Prediction	Each classifier votes for a class, and the class with the **most votes** is selected.	The classifier with the **highest probability** output (confidence score) determines the class.
Example: For a 3-class problem with classes 1, 2, and 3:	Classifier 1: Distinguish between class 1 and 2. Classifier 2: Distinguish between class 1 and 3. Classifier 3: Distinguish between class 2 and 3.	Classifier 1: Distinguish class 1 and not-1 (2, 3). Classifier 2: Distinguish class 2 and not-2 (1, 3). Classifier 3: Distinguish class 3 and not-3 (1, 2).

Note: K is the number of classes.

One-vs-One – Example

Given a set of features describing an animal (e.g., x_1 weight, x_2 height),

Predict whether it is a cat, dog, or rabbit.

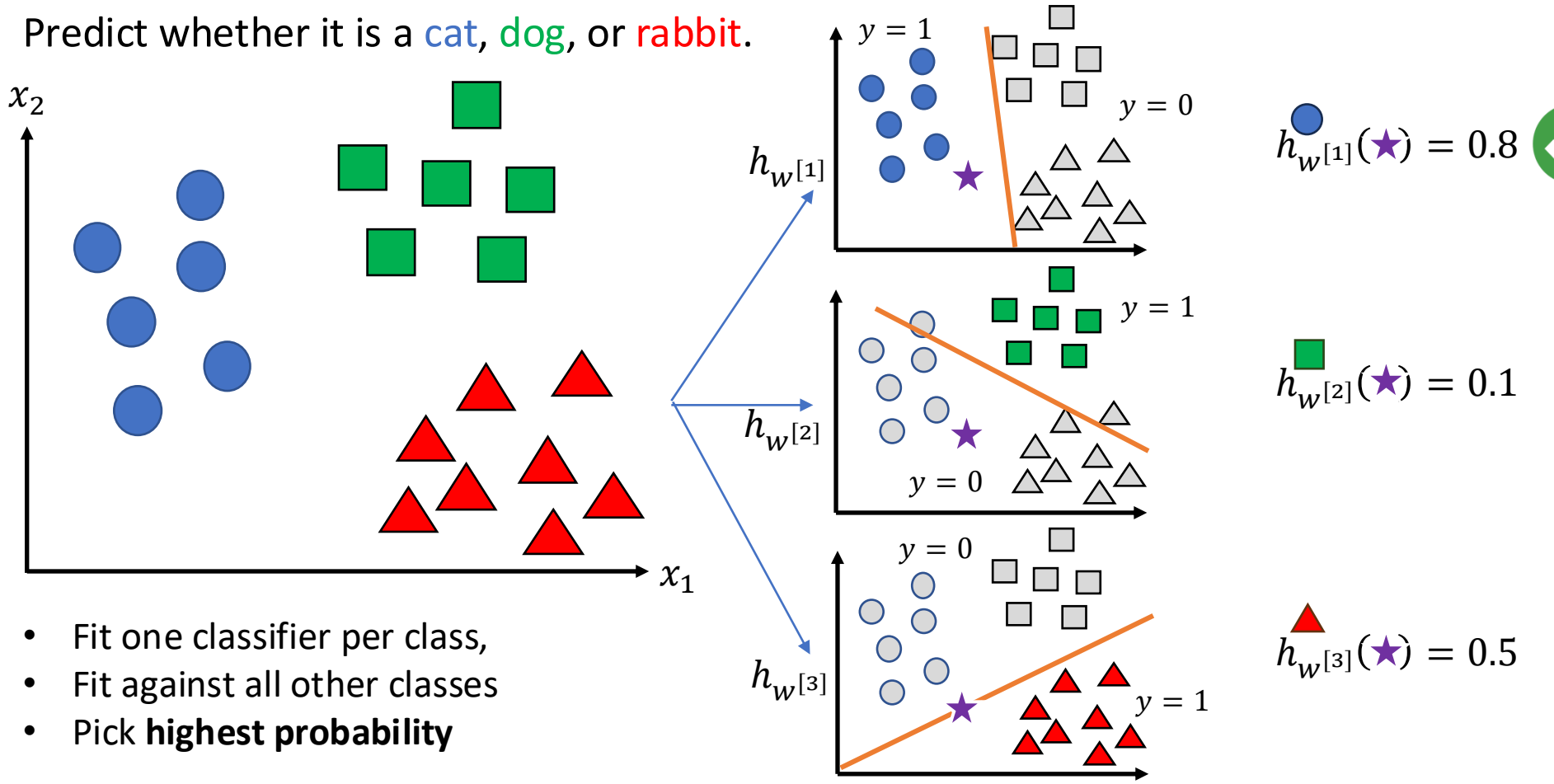


- Fit one classifier per class pair
- Pick **most wins**

One-vs-Rest – Example

Given a set of features describing an animal (e.g., x_1 weight, x_2 height),

Predict whether it is a cat, dog, or rabbit.



- Fit one classifier per class,
- Fit against all other classes
- Pick **highest probability**

Outline

- Logistic Regression
  - Data
  - Model
  - Loss
- Learning via Gradient Descent
- Feature Transformation
- Multi-Class Classification
- **Multi-Label Classification**
- Advanced Topics in Supervised Learning
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Multi-label: Examples

So far, labels are mutually exclusive: It's a cat and not a rabbit or dog.

However, often one is interested in multiple, non-exclusive labels for a data point.

- Semantic scene classification: "field scene with mountain in the background", "beach in urban environment".
- News article categorization: Politics, finance, …
- Medical diagnosis: patients with diabetes, high blood pressure, …

Mountain ✓	Field ✓	Building ✗	Beach ✗

Mountain ✗	Field ✗	Building ✓	Beach ✓

Multi-label Classification

Suppose:

- We are given n data points.
- Each data point consists of features and a target variable.
- Each data point has d features and they are described by real numbers.
- The target is $\{0,1\}^K$ where K is the number of classes.
  - The target is a subset of labels/classes (or an indicator vector of relevant labels).

Suppose we are given another data point $x \in \mathbb{R}^d$ and no target. Based on the dataset, find a model that predicts the target $y \in \{0,1\}^K$ for that x.

How to do this?

Multi-label: Common Techniques

- Transformation to binary classification problem
  - **Binary Relevance**: Train one independent binary classifier per label (One-vs-Rest). Ignores label correlations and constraints (e.g., co-occurrence, mutual exclusivity).
    Example: scene classifier → mountain classifier, field classifier.

    The idea of relevance can be extended to **non-binary** classifiers.
    Example: medical diagnosis → diabetes level [1,2,3,4], blood pressure [low, normal, high].

  - **Classifier Chain**: Like Binary Relevance but feed predictions of previous labels as features for subsequent labels. Captures label dependencies, but sensitive to chain order.

- Transformation into multi-class classification problem
  - **Label Powerset**: Treat each unique label-set as a single multiclass label. Preserves correlations but can explode in classes and be data-sparse.
    Example: scene classifier predicts “mountain”, “field”, “mountain and field”, …

Outline

- Logistic Regression
  - Data
  - Model
  - Loss
- Learning via Gradient Descent
- Feature Transformation
- Multi-Class Classification
- Multi-Label Classification
- **Advanced Topics in Supervised Learning**
  - Generalization, Dataset, Model Complexity
  - Overfitting & Underfitting
  - Hyperparameter Tuning

Generalization

- **Recall:** In supervised learning, and machine learning in general, the model's performance on unseen data is all we care about. This ability to perform well on new, unseen data is known as the model's generalization capability.
- Here, we assume that all the training and test data comes from the same unknown **ground truth**, i.e., true mapping from x to y.
- Measuring a model's error (e.g., MSE, MAE) is a common practice to quantify the performance of the model. This error, when evaluated on unseen data, is known as the generalization error.
- There are two factors that affect generalization: **dataset** (quality and quantity) and **model** complexity.

Dataset Quality

- **Relevance**: Dataset should contain relevant data, i.e., features that are relevant for solving the problem.
- **Noise**: Dataset may contain noise (irrelevant or incorrect data), which can hinder the model's learning process and reduce generalization.
- **Balance** (for classification): Balanced datasets ensure that all classes are adequately represented, helping the model learn to generalize well across different classes.

Basically, Garbage in → Garbage out

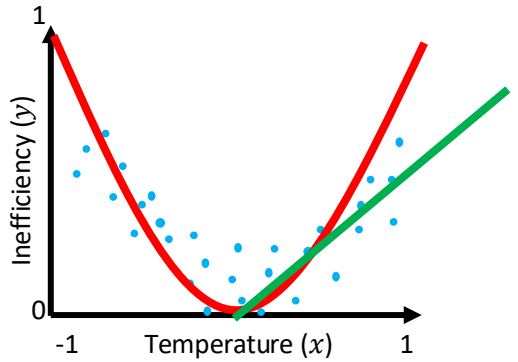
Dataset Quantity

In general, having more data typically leads to better model performance, provided that the model is expressive enough to accurately capture the underlying patterns in the data.

Extreme case: if the dataset contains every possible data point, the model does not need to "guess" or make predictions. Instead, it would only need to simply memorize all the data!

Model Complexity

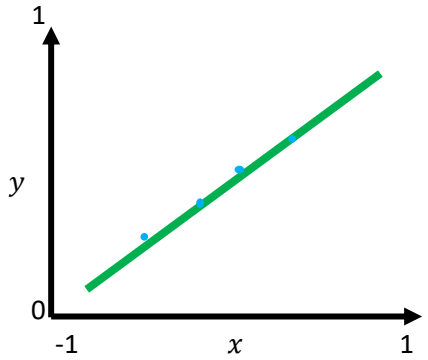
- Refers to the **size** and **expressiveness** of the model functions.
- Indicates how intricate the relationships between input and output variables that the model can capture.
- Higher model complexity allows for more sophisticated modeling of input-output relationships.
  - Example: polynomial regression model has a higher model complexity than linear regression model, thus it can model more complicated data.

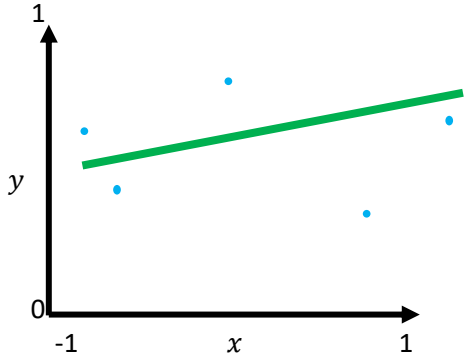


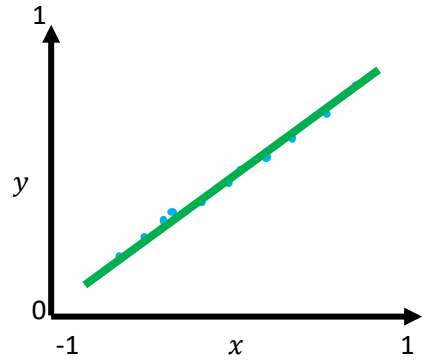
Case Study with Simple Model

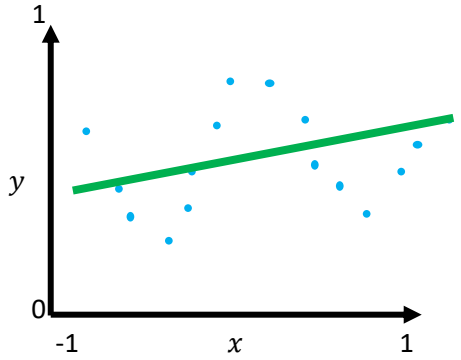
More **complex** ground truth

More data **quantity**









Simple model is good for simple ground truth – More data points not necessary.

Simple model is bad for complex ground truth – More data points do not help – In ML, we say the model has a **high bias**.

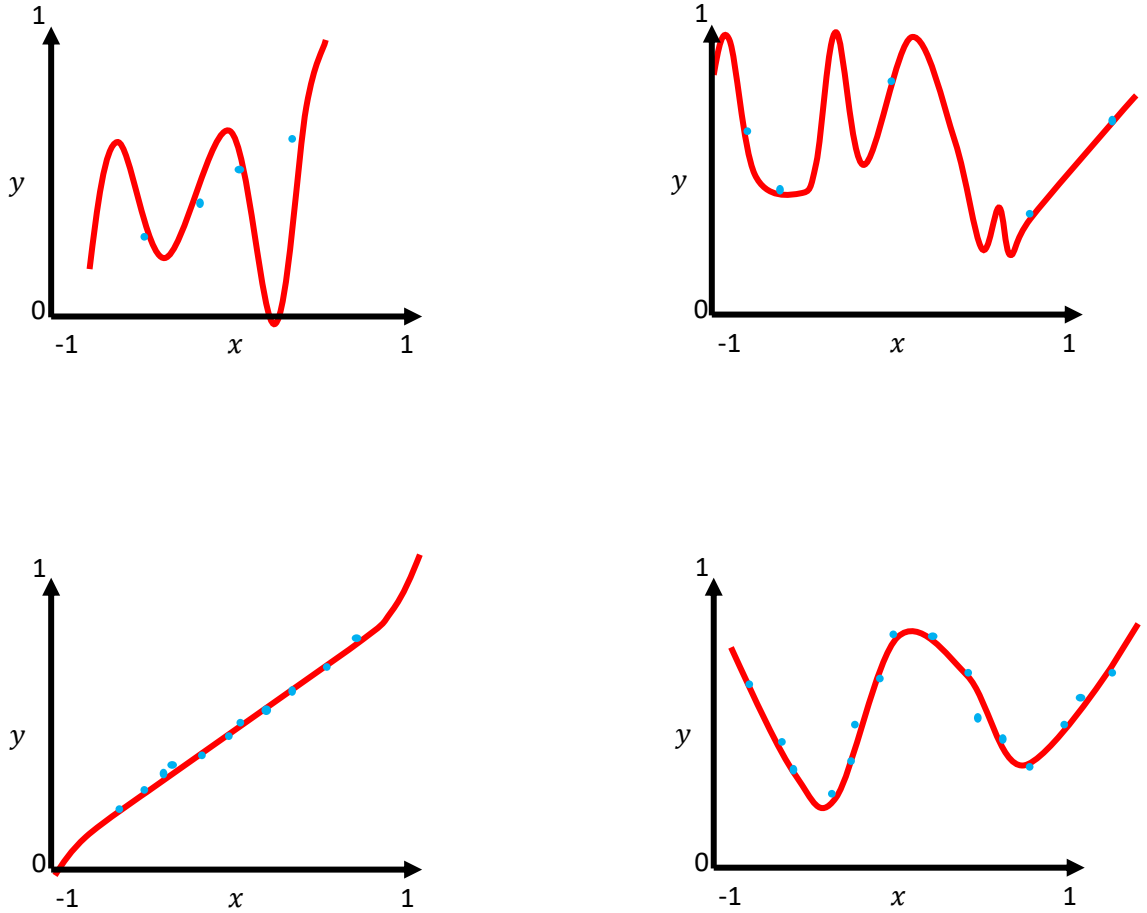
This phenomenon is called **underfitting**.

Retraining simple model with a different training set sampled from the same ground truth leads to essentially the same model – In ML, we say the model has a **low variance**.

Case Study with Complex Model

More **complex** ground truth

More data **quantity**



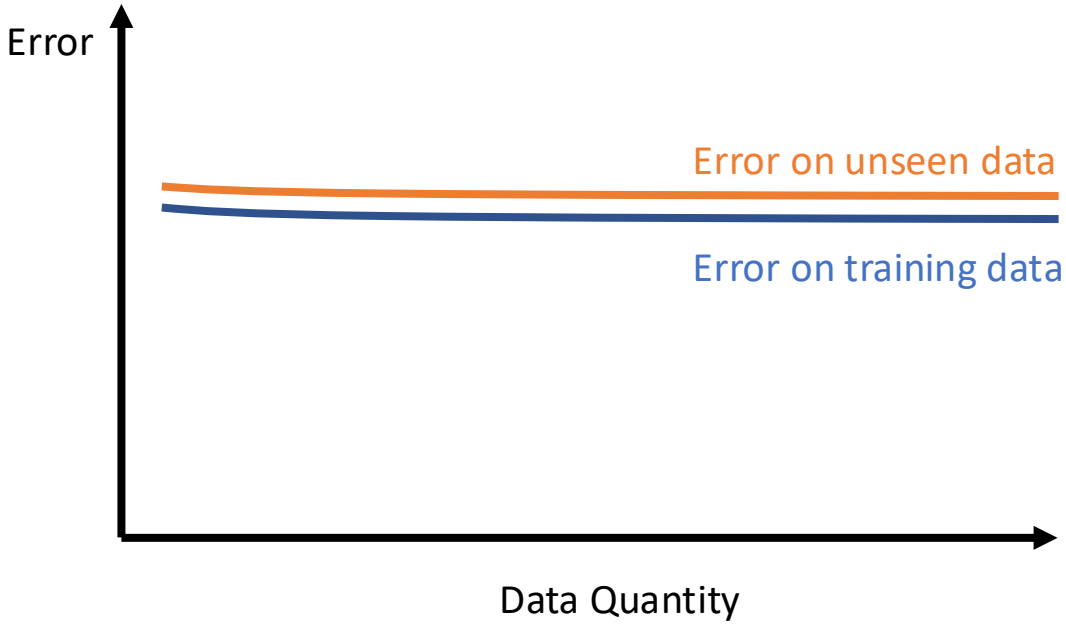
When data is scarce, complex models **overfit** the training data, potentially leading to incorrect predictions.

When data is abundant and noise is minimal, complex models can fit both simple and complex data. The model has **low bias**.

Retraining a complex model with a different training set from the same ground truth can lead to a vastly different model, exhibiting **high variance**.

Model Complexity and Data Quantity vs Error

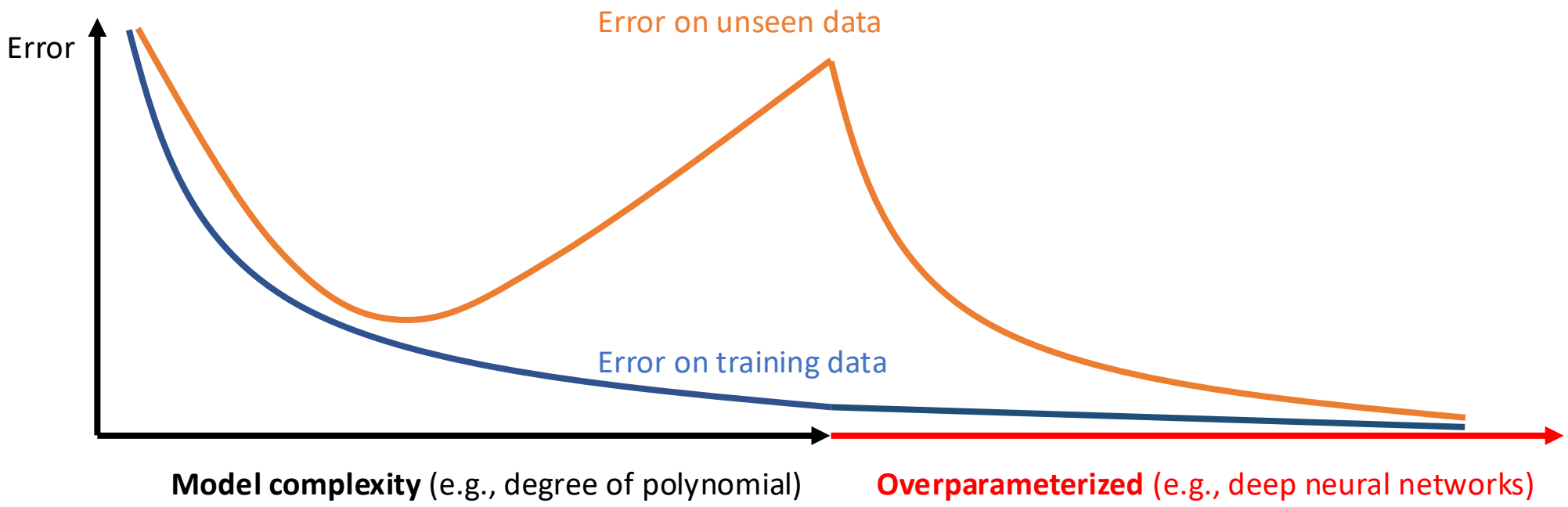
High-complexity Model



Low-complexity Model

Model Complexity vs Error

on limited number of data



Hyperparameters

- **Hyperparameters** are settings that control the behavior of the training algorithm and model but are not learned from the data. They need to be set before the training process begins. Example:
  - Learning rate (e.g., 0.1, 0.5, …)
  - Feature transformations (e.g., polynomial degree)
  - Batch size and iterations in mini-batch gradient descent
- **Hyperparameters** vs. **Parameters**:
  - Parameters are learned during training (e.g., weights in a linear model),
  - Hyperparameters are predefined and adjusted manually (e.g., learning rate).
- **Hyperparameter tuning/search** is the process of optimizing the hyperparameter of a machine learning model to improve its performance.
  - We need to use a separate evaluation data for tuning, called validation set.

Hyperparameter Tuning – Techniques

- **Grid search (exhaustive search)**
  - All possible combinations of a predefined set of hyperparameters are exhaustively tried.
  - **Example**: Trying all possible combinations of learning rates and regularization.
- **Random search**
  - Hyperparameters are randomly selected from a predefined distribution. Unlike grid search, it does not try every possible combination.
  - **Example**: Randomly sampling learning rates and regularization parameters.
- **Local search**
  - Use local search algorithms, such as hill-climbing, to iteratively optimize the hyperparameters. It starts with an initial set of hyperparameters and makes incremental changes to improve performance.
  - **Example**: Starting with an initial learning rate and regularization parameter and iteratively adjusting them to improve model performance using hill-climbing.
- **Successive halving**, **Bayesian optimization**, …

Many “off-the-shelves” packages available (e.g., in scikit-learn, hyperopt)!

Summary

- Logistic Regression: compute the probability of an input belonging to a class
  - Model: d dimensional input features: $h_w(x) = \sigma(\sum_{j=0}^{d} w_j x_j) = \sigma(w^T x)$
  - Binary Cross Entropy Loss:
    - $\frac{1}{N}\sum_{i=1}^{N} BCE\left(h_w\left(x^{(i)}\right), y^{(i)}\right)$, $BCE(y, \hat{y}) = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$
  - Non-linearly separable data: use feature transformations
- Learning via Gradient Descent: derivative is the same with linear regression!
- Multi-Class Classification:
  - One-vs-one: create a classifier for each pair of classes, pick most votes
  - One-vs-rest: create a classifier for each class vs the rest of the classes, pick the highest output
- Multi-Label Classification: convert to binary/multi-class problem
- Advanced Topics in Supervised Learning
  - Generalization: performance on unseen data
  - Model Complexity: how expressive the model is, e.g., polynomial degree
  - Overfitting & Underfitting: fit too much to training data vs can't fit even the training data
  - Hyperparameter Tuning: optimize the configuration of model and training

Further Reading (Optional)

- **History:** Historical use of logistic regression to investigate the root cause of the Challenger explosion in 1986.
  - (For example: https://cooperrc.github.io/data-driven-decisions/module_03/challenger.html)
- **Choosing the best model:** Model selection, cross-validation (AIAMA, Chapter 18.4)
- **Bias-Variance decomposition.** (PRML, Chapter 3.2)

To Do

- **Lecture Training 6**
  - +550 EXP
  - +100 Early bird bonus
- **Problem Set 3**
  - Will be released later today!